

Seeds, and How a Minimum-Time Transfer Is Actually Computed

The state–costate trajectory that starts a shooting solve: what it is,
where it comes from, and why a bare $\lambda(0)$ will not do

M. Casey (Coorbital, `orbit_transfer/DR0_tulip`)

2026-09-15

Abstract

Everyone who reads this code asks the same question: *what is a seed?* The short answer is that a seed is not a guess at the initial costates — it is an entire state-and-costate *trajectory*, sampled at the junction times of a multiple-shooting mesh, and the reason it must be a trajectory is a measured fact about this problem, not a stylistic preference. This document explains the seed, the three ways the library builds one, the full route from a discrete-time direct solve to a certified catalog entry, and the economics that decide which route a given entry takes. Each topic is given three times: as a picture (ELI5), as the reasoning needed to trust it (Intuition), and as the equations and code contracts (Rigor), with the campaign’s measurements called out where they settled a question. Companion documents: `doc/algorithms_orbit_transfer.tex` (the OCP and both objective lines), `DR0_tulip/doc/phase_torus_methods.tex` (how a whole library is built), `DR0_tulip/process/PHASE_TORUS_RUNBOOK.md` (the operational recipe).

Contents

1	The problem, in one page	1
2	Why a bare $\lambda(0)$ does not work	2
3	What a seed is	3
4	Where seeds come from: three constructions	4
4.1	(1) Harvest from a direct solve	4
4.2	(2) Re-cut an existing root	5
4.3	(3) The previous converged root (continuation)	5
5	The route, end to end	6
5.1	The gate stack: where a number becomes an entry	6
5.2	The homogeneous chart	7
6	Economics: which route each entry took	8
7	A worked reading of the code	9
8	Summary	9

1 The problem, in one page

ELI5

A spacecraft is coasting around the Moon on one closed loop (a distant retrograde orbit, the *DRO*). We want it on a different closed loop (a *tulip* orbit, which looks like a flower with seven petals). The engine is weak — 70 millinewtons, about the weight of a grape on Earth — so the trip takes weeks and the engine is on the whole time. We want the fastest possible trip. The question this document answers is: how do you even *start* looking for that trip?

Intuition

Minimum-time with a bounded thrust that is always worth using is a *bang* problem: the optimal engine setting is always full throttle, and the only thing to decide is which way to point. Pontryagin’s principle turns “which way to point, at every instant” into “pick seven numbers at the start” — the initial costates — because the pointing direction at any later time is determined by integrating a differential equation from those seven numbers. So an infinite-dimensional control problem becomes a root-finding problem in $\mathbb{R}^8$: seven initial costates and the final time. That is the good news. The bad news is the whole of §2.

Rigor

State $x = (\mathbf{r}, \mathbf{v}, m) \in \mathbb{R}^7$ in the rotating CR3BP frame, mass ratio μ , thrust magnitude T , exhaust speed c :

$$\dot{\mathbf{r}} = \mathbf{v}, \quad \dot{\mathbf{v}} = \mathbf{g}(\mathbf{r}, \mathbf{v}) + \frac{T}{m} \mathbf{u}, \quad \dot{m} = -\frac{T}{c}, \quad \|\mathbf{u}\| = 1.$$

Costates $\lambda = (\lambda_r, \lambda_v, \lambda_m)$, Hamiltonian $H = 1 + \lambda_r^\top \mathbf{v} + \lambda_v^\top (\mathbf{g} + \frac{T}{m} \mathbf{u}) - \lambda_m \frac{T}{c}$. Minimising H over $\|\mathbf{u}\| = 1$ gives the *primer vector* law

$$\mathbf{u}^* = -\frac{\lambda_v}{\|\lambda_v\|},$$

and the adjoint equations $\dot{\lambda} = -\partial H / \partial x$ close the system. Free final time with a fixed target state gives the transversality conditions $\lambda_m(t_f) = 0$ and $H(t_f) = 0$. The unknown is

$$z = (\lambda(0), t_f) \in \mathbb{R}^8,$$

and a *root* is a z whose flown trajectory hits the target state with $\lambda_m(t_f) = 0$. Endpoints are parameterised by phase: the departure state is $\mathbf{x}_D(s_D)$ and the arrival state $\mathbf{x}_A(s_A)$, with $s_D, s_A \in [0, 1)$ fractions of the two orbits’ periods — which is what makes a *library* over a (s_D, s_A) grid meaningful.

2 Why a bare $\lambda(0)$ does not work

ELI5

Imagine aiming a rifle at a target forty kilometres away, except the bullet loops around the Moon forty times on the way. A hair's difference in aim at the muzzle becomes a miss of thousands of kilometres. Having the right seven numbers to eight decimal places is not enough, because the computer's arithmetic — and the solver's first guess — are never that good.

Intuition

The min-time extremal is unstable in the forward direction: small perturbations of $\lambda(0)$ grow roughly exponentially along the arc, because the linearised dynamics of the state–costate (Hamiltonian) system have eigenvalues in pairs $\pm\sigma$ and the arc is long — tens of revolutions. A solver that only knows $\lambda(0)$ has to shoot the whole arc in one go (“single shooting”), so its residual is an astronomically amplified function of its unknowns: the Jacobian is enormous, the basin of convergence is microscopic, and Newton steps are meaningless.

The cure is standard and old: *multiple shooting*. Cut the arc into K short pieces. Now no single propagation runs long enough for the amplification to hurt. The price is that the junction states between pieces become unknowns too — so the solver needs an initial guess not just for $\lambda(0)$, but for the entire state and costate history. *That guess is the seed.*

Measured

This was measured on this problem, not assumed. Flying the PMP equations forward from a catalog entry's stored $\lambda(0)$ misses the target by **36,000 to 560,000 km**, while the same entry's recorded control history lands within 10km (memory note `direct-duals-not-shooting-seeds`; the header of `costate_common/ms_tfmin.m` records the $\sim 10^3 \times$ Lyapunov amplification that explains it). It is also why the campaign's binding rule is *ship full junction states, not bare z_8* (`flagship-tworoot-basin`): a bare 8-vector is not enough information to reconstruct its own trajectory reliably.

Rigor

Multiple shooting with K segments and junction times $0 = t_0 < t_1 < \dots < t_K = t_f$ introduces unknowns $Y_k \in \mathbb{R}^{14}$, $k = 0, \dots, K$, where $Y = (x; \lambda)$, and imposes *continuity defects*

$$R_k = \varphi(Y_{k-1}; t_{k-1} \rightarrow t_k) - Y_k = 0, \quad k = 1, \dots, K,$$

with φ the flow of the Hamiltonian system, together with the boundary conditions (departure state fixed, arrival state matched, $\lambda_m(t_f) = 0$, $H(t_f) = 0$). The Jacobian is block-bidiagonal: each block is a state-transition matrix over *one short segment*, whose condition number is the K -th root of the single-shooting one. The campaign uses $K = 24$.

3 What a seed is

ELI5

A seed is a complete rough draft of the answer: where the spacecraft is, how fast it is going, how much fuel it has left, and what the seven “steering” numbers are — at 25 moments spread across the trip. The solver’s job is to take that rough draft and make it exactly consistent.

Intuition

It helps to see the seed as a *discretised trajectory in the doubled space*. The physical trajectory lives in $\mathbb{R}^7$; the extremal lives in $\mathbb{R}^{14}$, states and costates together, because Pontryagin’s principle is a statement about that doubled space. A seed is a sampling of a curve in $\mathbb{R}^{14}$ — one that is nearly, but not exactly, an integral curve of the Hamiltonian field. “Nearly” is enough: the solver only has to remove the small discontinuities at the junctions.

Rigor

The contract (`costate_common/ms_tfmin.m`, lines 20–33):

field	size	meaning
<code>seed.tf</code>	scalar	time of flight, nondimensional
<code>seed.tGrid</code>	$1 \times (K+1)$	junction times $0, \dots, t_f$
<code>seed.Y</code>	$14 \times (K+1)$	$[\mathbf{r}; \mathbf{v}; m; \lambda_r; \lambda_v; \lambda_m]$ at those times

Column 1’s *state* part is overwritten with the true departure state $[\mathbf{x}_D(s_D); 1]$, so only the costates are free there; the remaining columns are free in all 14 components. The solver returns $z = (\lambda(0), t_f)$ in exactly `pumpkyn.cr3bp.tfMin`’s convention, so a root from multiple shooting is interchangeable with one from the original single-shooting code.

Where it breaks / what is assumed

A seed is *not* a certificate and not a solution. It is an initial iterate. Every quantity a seed carries is overwritten by the solve; nothing from a seed reaches a catalog entry except through a converged root that then passes the gate stack (§5.1).

4 Where seeds come from: three constructions

4.1 (1) Harvest from a direct solve

ELI5

First solve a cruder, easier version of the problem: chop time into 800 pieces and let a big optimiser push all the positions and thrust directions around until they are consistent and the trip is short. That optimiser, as a side effect of how it works, computes exactly the seven steering numbers we need at every step — we just have to know where to look and what

sign convention it used.

Intuition

This is the deep and useful fact: **the Lagrange multipliers of a direct method’s dynamics constraints are a discretisation of the costates**. The direct method never mentions Pontryagin; it just minimises t_f subject to “the trajectory obeys the equations of motion”. But the multiplier attached to each such constraint measures how much the objective would improve if that constraint were relaxed — which is the definition of the adjoint variable. So a direct solve hands us, free of charge, the costate history we could not otherwise guess.

Two cautions turn this from a slogan into working code. First, the multipliers of a Hermite–Simpson scheme are attached to *segment midpoints*, not to the nodes — pairing them with nodes silently shifts the costates half a step, which is the kind of error that still converges and gives a slightly wrong answer. Second, the sign convention depends on how the NLP was written; it must be determined, not assumed.

Rigor

`casadi_mintime_dro` solves the NLP: Hermite–Simpson collocation with $N = 800$ intervals, a Sundman-regularised independent variable (so the mesh follows the geometry, not the clock), the 1900 km lunar clearance as a path constraint, IPOPT as the solver. Its output carries `o.X` (states at nodes), `o.Um` (solved thrust directions), `o.tf`, and `o.lamDef` — the KKT multipliers of the defect constraints.

`harvest_ms_seed(o,K)` then:

1. calls `oc.duals_to_costates` with the scheme name, the multiplier array, the node times and the solved thrust directions. That function owns the subtle rules: midpoint station association for Hermite–Simpson, a *sign vote* across the arc (the primer must oppose λ_v at every station; the majority sign wins and the margin is reported), and — when the NLP exposes it — the check that the multiplier of the time-scaling constraint comes out $\lambda_t = +1$;
 2. interpolates the states onto the $K+1$ junction times with `pchip`;
 3. interpolates the costates onto the same times with `pchip` and *extrapolation*, because midpoint stations do not reach $t = 0$ or $t = t_f$;
 4. returns `seed = (tf,tGrid,Y)` and a diagnostics struct (sign, vote margin, λ_t , notes).
- One home per subtle rule: the mapping lives in the cross-folder library `oclib/+oc/duals_to_costates.m` because a second consumer (`booster_landing`’s primer gate) independently re-invented — and independently re-fixed — the same midpoint bug.

4.2 (2) Re-cut an existing root

Intuition

If a root already exists, its own trajectory is the best possible seed for a nearby problem: fly it and slice it. This is how a stored 8-vector is turned back into a 14-row trajectory, and how a solve is moved to a different mesh size without re-solving from scratch.

Rigor

`seed_from_z8(z, rv0, K, Tnd, cnd, mu)` propagates the PMP flight with `pumpkyn.cr3bp.tfMinProp` from $[x_D; 1; \lambda(0)]$ over $[0, z_8]$ and samples it at $K+1$ times. `seed_from_entry` prefers an entry's stored junction states `.Y` when it has them and falls back to `seed_from_z8` when it does not — which is precisely why entries ship their junctions.

4.3 (3) The previous converged root (continuation)

ELI5

Once you have solved one transfer, the transfer to a slightly different arrival phase is almost the same trip. Start from the answer you already have and nudge. This is how nearly all of the library was built — not by solving each cell from scratch.

Intuition

Formally the roots form curves in $(\text{parameter}, z)$ space; numerically, walking along such a curve is the cheapest possible seeding, because the distance between consecutive seeds is a step size you control. Two axes, two walkers:

- **arrival phase** — pseudo-arclength continuation, because this axis has *folds*: places where the branch turns back in s_A , so no function $s_A \mapsto z$ exists there and a naive parameter sweep dies. Arclength continuation walks the curve itself.
- **departure phase** — a plain bisectioning walker, because departure phase enters only through the first junction's fixed state, so steps are cheap and the curve is tame. A failed step halves the stride (up to eight times) and retries.

Rigor

Pseudo-arclength: at a root (q_i, p_i) (with $q = s_A$, p the vector of all shooting unknowns) compute the tangent τ to the solution curve from the Jacobian, predict $w^{(0)} = (p_i, q_i) + \Delta s \tau$, and correct with Newton *in the hyperplane orthogonal to τ* :

$$\begin{cases} R(p, q) = 0 \\ \tau^\top [(p, q) - w^{(0)}] = 0. \end{cases}$$

Δs adapts to the Newton count. A sign change in the q -component of τ localises a fold; the normality scalar ρ of the homogeneous chart (§5.2) is monitored alongside, because losing normality is a different kind of end than folding. Each crossing of a grid level is re-solved at exactly that level before certification — never interpolated.

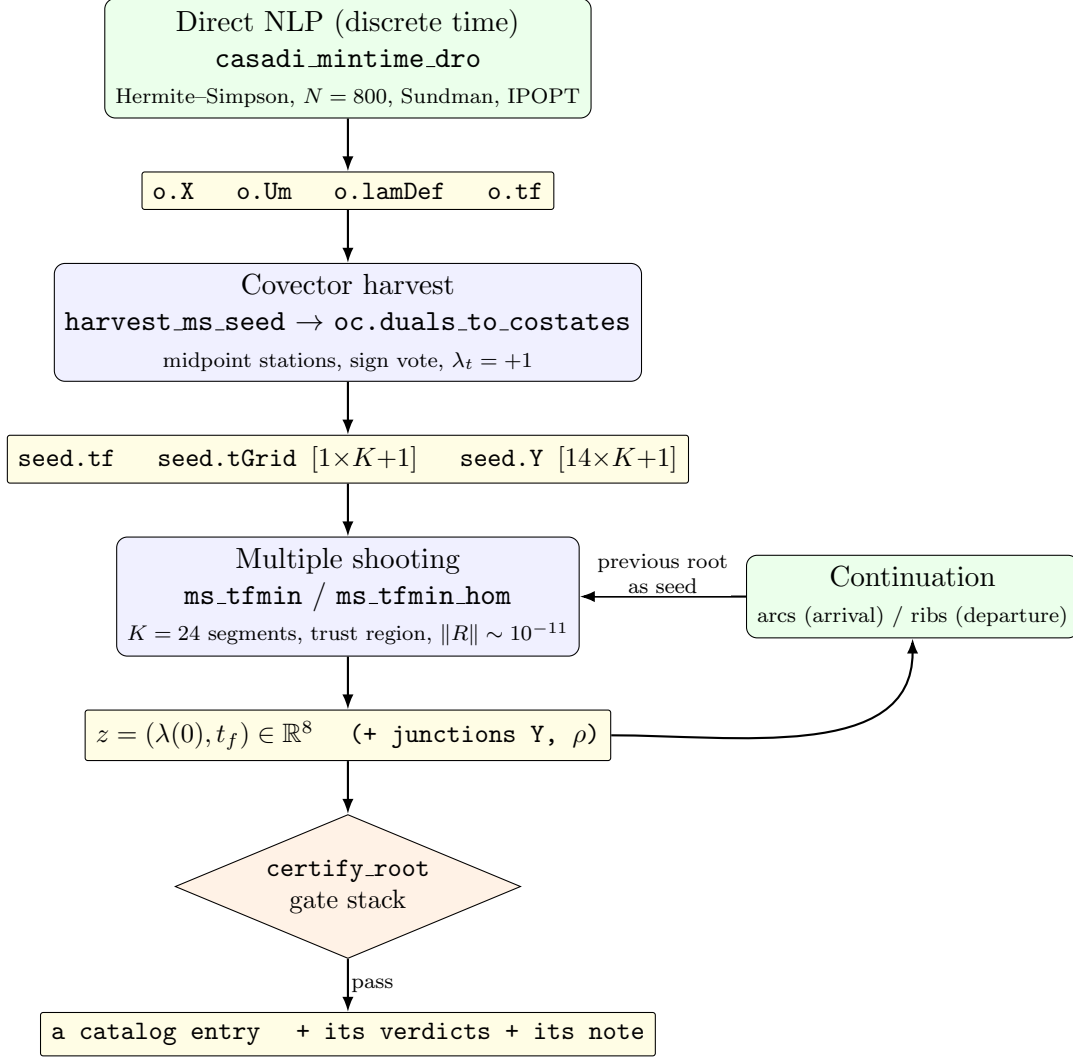


Figure 1: The two ways into the shooting solve. The direct branch (left) pays for a seed when no neighbour exists; the continuation branch (right) reuses the previous converged root and is what produced most of the library.

5 The route, end to end

5.1 The gate stack: where a number becomes an entry

Intuition

A converged root is not yet an answer. Newton’s method converges to *stationary points*, and a stationary point of a shooting residual can be a maximum, a saddle, a solution of a different problem, or a numerical artefact. The gate stack is the campaign’s answer to “how do you know?”, and it is deliberately redundant: the flight is re-integrated from z alone at tight tolerance, so a root that only looks converged in the solver’s own arithmetic is caught.

Rigor

`certify_root` applies, in order, and stops at the first failure with a named reason:

1. multiple-shooting residual below `tolR` (a polish that plateaus is allowed through but *says so* in the entry’s reason and note);
2. the flown arrival: re-integrate from z with `ode113` at 10^{-13} and require the miss in *position and velocity* below gates;
3. transversality $\lambda_m(t_f) = 0$ read off that tight flight (reading it off a loose flight cost the campaign five certified phases — FINDINGS §59);
4. pointwise PMP checks along the arc: $\max |H|$, adjoint-equation residual, the minimum-principle gap (is the applied control actually the minimiser?), throttle = 1;
5. a `pumpkyn tfMin` witness — an independent solver reaching the same root;
6. the free-time conjugate test: the quotiented Jacobi determinant on a dense scan, no interior sign change (this is the Bonnard–Caillau–Trélat second-order test, and it refuted 12 of 14 candidates that passed every first-order check — FINDINGS §38);
7. the BCT sufficiency hypotheses: strong Legendre $\min_t \|\lambda_v\| > 0$, $Q_{mt} > 0$, and normality via $\dim S = 1$ with a $10\times$ margin;
8. lunar clearance ≥ 1900 km along the whole flight.

5.2 The homogeneous chart

Intuition

The costates of a min-time problem are defined only up to positive scaling in the *abnormal* case, and near a loss of normality the ordinary normalisation ($\lambda_t = 1$, say) becomes singular — the solver’s unknowns run off to infinity while the trajectory stays perfectly finite. Carrying an extra scalar ρ and normalising (λ, ρ) onto a sphere removes the singularity: the walk passes smoothly through the place where the ordinary chart would have died, and $\rho \rightarrow 0$ becomes a *diagnosis* (“this family is losing normality here”) rather than a crash.

Rigor

`ms_tfmin_hom` solves the same boundary-value problem with the homogeneous unknown (λ, ρ) normalised, and `ms_bvp` carries `nExtra` such scalars generically. In the arcs, ρ is recorded at every step; its minimum over a family and the phase where it occurs are part of the family map, and `certify_root` uses the normal chart for the final polish so the shipped z is in `tfMin`’s convention.

6 Economics: which route each entry took

ELI5

Solving from scratch is expensive; nudging a known answer is cheap. So we solve from scratch only when we must — to start a new family, or to fill a cell nothing else can reach — and nudge everywhere else.

Rigor

Of the 576 certified entries of the 70 mN 24×24 library:

route	entries	seed
rib continuation (departure axis)	503	the previous accepted rib point's junctions
arc crossing / registered root (spine)	24	the previous root on the arc, or a stored root
direct cell solve	49	a certified neighbour's PMP flight

Costs: a rib point takes 2–10 minutes; an arc of 4000 steps takes 2–6 hours and yields tens of crossings; a direct cell solve takes 1–5 minutes when it converges and burns its whole cap (300–900s) when it does not.

Measured

Seed geometry decides everything. In the hole-filling pass, seeding a direct solve from the neighbouring *column* (a root at a different arrival phase, however fast) sent IPOPT to 100–400-day junk, while seeding from the same column's neighbour (same arrival geometry, adjacent departure phase) converged in 1–3 minutes. The filler now tries same-column neighbours first and caps the solve at 300s.

And it decides across problems too. Asked for the first root of a *different* problem — DRO period $1 \rightarrow 2$ and $7 \rightarrow 8$ petals — a direct solve warm-started from the 7-petal 70 mN root returned a 259-day trajectory passing through the centre of the Moon. Changing only the petal count ($\tau = 1$, $7 \rightarrow 8$) still failed: 103 days, again through the Moon. The arrival orbit's geometry, not the period, is what the warm start cannot bridge. A new orbit pair therefore needs its own anchor route — a thrust ladder from a shipped catalog entry at the same orbit pair, or a structured cold guess — not a warm start from a different pair.

Where it breaks / what is assumed

A direct solve is *branch-blind*: it lands in whatever basin its seed is near, which is a feature when you are hunting for an unknown family and a hazard when you assume it returns the family you started from. Four of the library's five families were found precisely by exploiting this — solving at a phase where the known family was slow, warm-started from that known family, and discovering the solver had landed somewhere else entirely, 5–7 days faster. Continuation would never have found them: it follows its own sheet, faithfully, forever.

7 A worked reading of the code

Rigor

The shortest complete path through the library, for one transfer at (s_D, s_A) :

1. `arclength.arrival('setup', so)` builds the physics closures B for the operating point: the two orbits' state functions $\mathbf{x}_D(s_D)$, $\mathbf{x}_A(s_A)$, the nondimensional thrust and exhaust speed, the residual and its parameter derivative. With `.physicsOnly` it stops

there — for a problem that has no root yet.

2. A seed by one of §4’s three routes.
3. `ms_tfmin(rv0, rvf, seed, Tmax, c, mu, opts)` → `z`, `info.Y` (junctions), `info.normR`; or `ms_tfmin_hom` for the homogeneous chart.
4. `certify_root(seed, rv0, rvf, B, copts)` → a verdict struct: `.ok`, `.reason`, `.note`, `z`, `Y`, `tf` in days, the flown misses, the conjugate verdict, the three gate margins, the lift margin, wall time.
5. Packaging (`sheet_to_catalog_file` → `build_costate_catalog_family`) carries only certified entries, with their verdicts, their family label and their provenance note.

For a single transfer the front door `run_dro_tulip(sD, sA)` does all of this and reports the route it took; for a whole library, `run_phase_torus(spec)`.

8 Summary

ELI5

A seed is a rough draft of the entire trip — positions and steering numbers at 25 moments. You get one either by solving a cruder version of the problem with a big optimiser (which hands you the steering numbers as a by-product), or by nudging an answer you already have. Then a careful solver makes the draft exact, and a stack of independent checks decides whether the result is really the fastest trip and not just an arithmetic coincidence.

a seed is	a $14 \times (K+1)$ state–costate trajectory plus t_f and the junction times
it is not	a guess at $\lambda(0)$; single shooting from one misses by 10^4 – 10^5 km
made from	a direct solve’s duals; an existing root’s flight; the previous root of a continuation
consumed by	<code>ms_tfmin</code> / <code>ms_tfmin_hom</code> , which return $z = (\lambda(0), t_f)$
validated by	<code>certify_root</code> : eight independent gates, each with a named failure
economics	503 of 576 library entries came from continuation, 49 from direct cell solves, 24 from arcs and stored roots
